Student Productivity & Expense Manager Using C

1. Introduction

This project is a simple console-based application developed using the C programming language. The main purpose of this system is to help students manage their daily tasks and track their expenses in an organized way. In student life, managing time and money is very important, and this system provides a basic solution for that.

The system also gives simple smart suggestions based on user activity, which helps users become more aware of their productivity and spending habits.

2. Objectives

The main objectives of this project are:

- To help students manage their daily tasks efficiently
- To track and monitor daily expenses
- To improve productivity and time management
- To provide basic intelligent suggestions based on user data

3. Features

- Add new tasks with deadlines
- View all tasks
- Mark tasks as completed
- Add daily expenses with category
- View all expenses
- Calculate total expense
- Provide smart suggestions (alerts and warnings)

4. Tools & Technologies

- Programming Language: C
- Compiler: GCC / Cxxdroid / CodeBlocks
- Platform: Console-based application

5. System Design

This system follows a menu-driven design, which makes it simple and user-friendly. Users can select different options from the menu to perform

various operations.

The system uses:

- **Structure** to store task and expense data
- **Array** to manage multiple records
- **Function** to organize the program into smaller modules

Flow of the system:

- Start the program
- Display menu options
- User selects an option
- System performs the operation
- Return to the main menu
- Exit when user selects 0

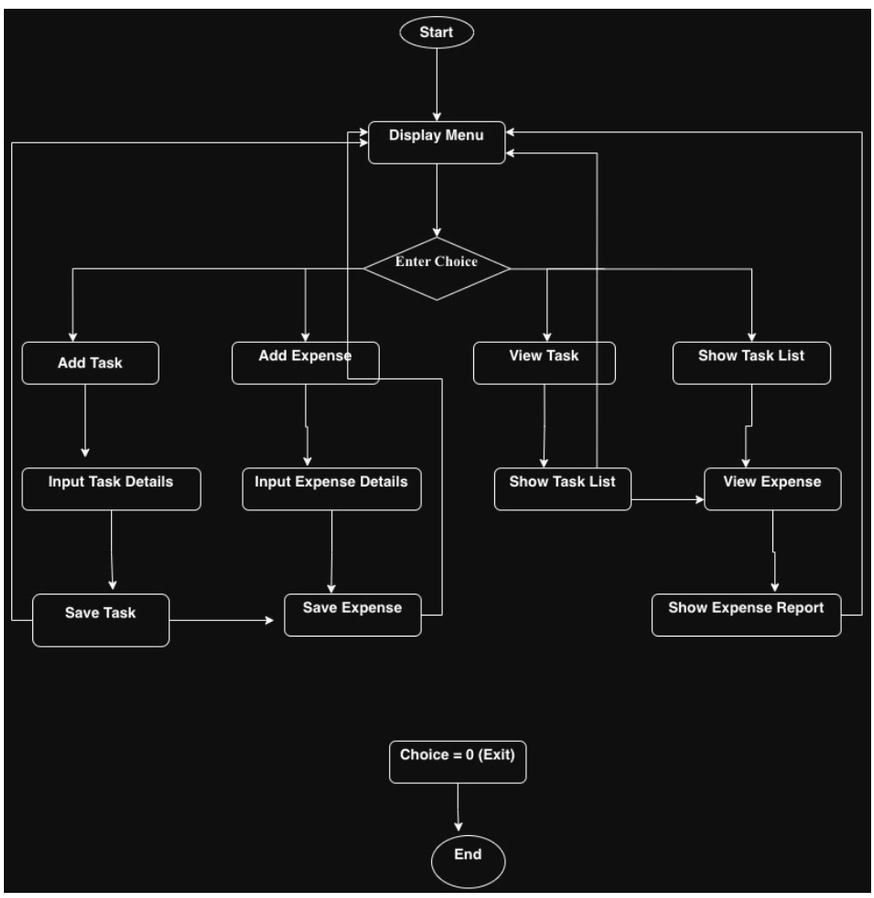


Figure 1: Flowchart of Student Productivity & Expense Manager

6. Implementation

Task Management

In this section, users can:

- Add tasks with deadlines
- View all added tasks
- Mark tasks as completed

This helps in organizing daily activities and tracking progress.

Expense Management

In this section, users can:

- Add expenses based on category (Food, Transport, etc.)
- View all expenses
- Calculate total spending

This helps users understand their spending patterns.

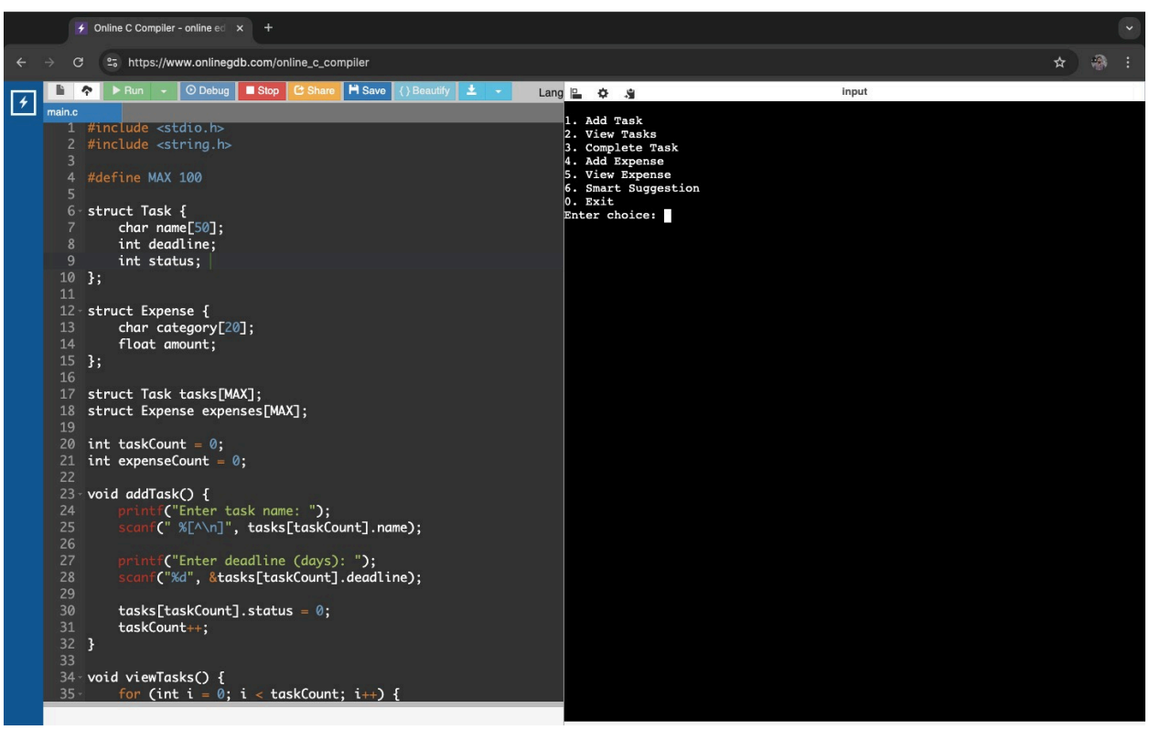
Smart Suggestion

The system provides simple suggestions based on user data:

- If total expense is high → Warning message
- If many tasks are pending → Alert message

This feature simulates basic intelligent decision-making.

7. Sample Input/Output



Online C Compiler - online ed

https://www.onlinegdb.com/online_c_compiler

Run | Debug | Stop | Share | Save | { } Beautify | Lang | Input

main.c

```c
#include <stdio.h>
#include <string.h>

#define MAX 100

struct Task {
    char name[50];
    int deadline;
    int status;
};

struct Expense {
    char category[20];
    float amount;
};

struct Task tasks[MAX];
struct Expense expenses[MAX];

int taskCount = 0;
int expenseCount = 0;

void addTask() {
    printf("Enter task name: ");
    scanf(" %[^\n]", tasks[taskCount].name);

    printf("Enter deadline (days): ");
    scanf("%d", &tasks[taskCount].deadline);

    tasks[taskCount].status = 0;
    taskCount++;
}

void viewTasks() {
    for (int i = 0; i < taskCount; i++) {
```

```
1. Add Task
2. View Tasks
3. Complete Task
4. Add Expense
5. View Expense
6. Smart Suggestion
0. Exit
Enter choice: 1
Enter task name: Assignment
Enter deadline (days): 4

1. Add Task
2. View Tasks
3. Complete Task
4. Add Expense
5. View Expense
6. Smart Suggestion
0. Exit
Enter choice:
```

Online C Compiler - online ed

https://www.onlinegdb.com/online_c_compiler

Run | Debug | Stop | Share | Save | { } Beautify | Lang | Input

main.c

```c
#include <stdio.h>
#include <string.h>

#define MAX 100

struct Task {
    char name[50];
    int deadline;
    int status;
};

struct Expense {
    char category[20];
    float amount;
};

struct Task tasks[MAX];
struct Expense expenses[MAX];

int taskCount = 0;
int expenseCount = 0;

void addTask() {
    printf("Enter task name: ");
    scanf(" %[^\n]", tasks[taskCount].name);

    printf("Enter deadline (days): ");
    scanf("%d", &tasks[taskCount].deadline);

    tasks[taskCount].status = 0;
    taskCount++;
}

void viewTasks() {
    for (int i = 0; i < taskCount; i++) {
```

```
1. Add Task
2. View Tasks
3. Complete Task
4. Add Expense
5. View Expense
6. Smart Suggestion
0. Exit
Enter choice: 1
Enter task name: Assignment
Enter deadline (days): 4

1. Add Task
2. View Tasks
3. Complete Task
4. Add Expense
5. View Expense
6. Smart Suggestion
0. Exit
Enter choice: 2
1. Assignment | Deadline: 4 | Pending

1. Add Task
2. View Tasks
3. Complete Task
4. Add Expense
5. View Expense
6. Smart Suggestion
0. Exit
Enter choice:
```

Online C Compiler - online ed

https://www.onlinegdb.com/online_c_compiler

Run | Debug | Stop | Share | Save | { } Beautify | Lang

main.c

```c
#include <stdio.h>
#include <string.h>

#define MAX 100

struct Task {
    char name[50];
    int deadline;
    int status;
};

struct Expense {
    char category[20];
    float amount;
};

struct Task tasks[MAX];
struct Expense expenses[MAX];

int taskCount = 0;
int expenseCount = 0;

void addTask() {
    printf("Enter task name: ");
    scanf(" %[^\n]", tasks[taskCount].name);

    printf("Enter deadline (days): ");
    scanf("%d", &tasks[taskCount].deadline);

    tasks[taskCount].status = 0;
    taskCount++;
}

void viewTasks() {
    for (int i = 0; i < taskCount; i++) {
```

input

```
1. Add Task
2. View Tasks
3. Complete Task
4. Add Expense
5. View Expense
6. Smart Suggestion
0. Exit
Enter choice: 4
Enter category: Food
Enter amount: 6000

1. Add Task
2. View Tasks
3. Complete Task
4. Add Expense
5. View Expense
6. Smart Suggestion
0. Exit
Enter choice: 5
Food: 6000.00
Total Expense: 6000.00

1. Add Task
2. View Tasks
3. Complete Task
4. Add Expense
5. View Expense
6. Smart Suggestion
0. Exit
Enter choice:
```

Online C Compiler - online ed

https://www.onlinegdb.com/online_c_compiler

Run | Debug | Stop | Share | Save | { } Beautify | Lang

main.c

```c
#include <stdio.h>
#include <string.h>

#define MAX 100

struct Task {
    char name[50];
    int deadline;
    int status;
};

struct Expense {
    char category[20];
    float amount;
};

struct Task tasks[MAX];
struct Expense expenses[MAX];

int taskCount = 0;
int expenseCount = 0;

void addTask() {
    printf("Enter task name: ");
    scanf(" %[^\n]", tasks[taskCount].name);

    printf("Enter deadline (days): ");
    scanf("%d", &tasks[taskCount].deadline);

    tasks[taskCount].status = 0;
    taskCount++;
}

void viewTasks() {
    for (int i = 0; i < taskCount; i++) {
```

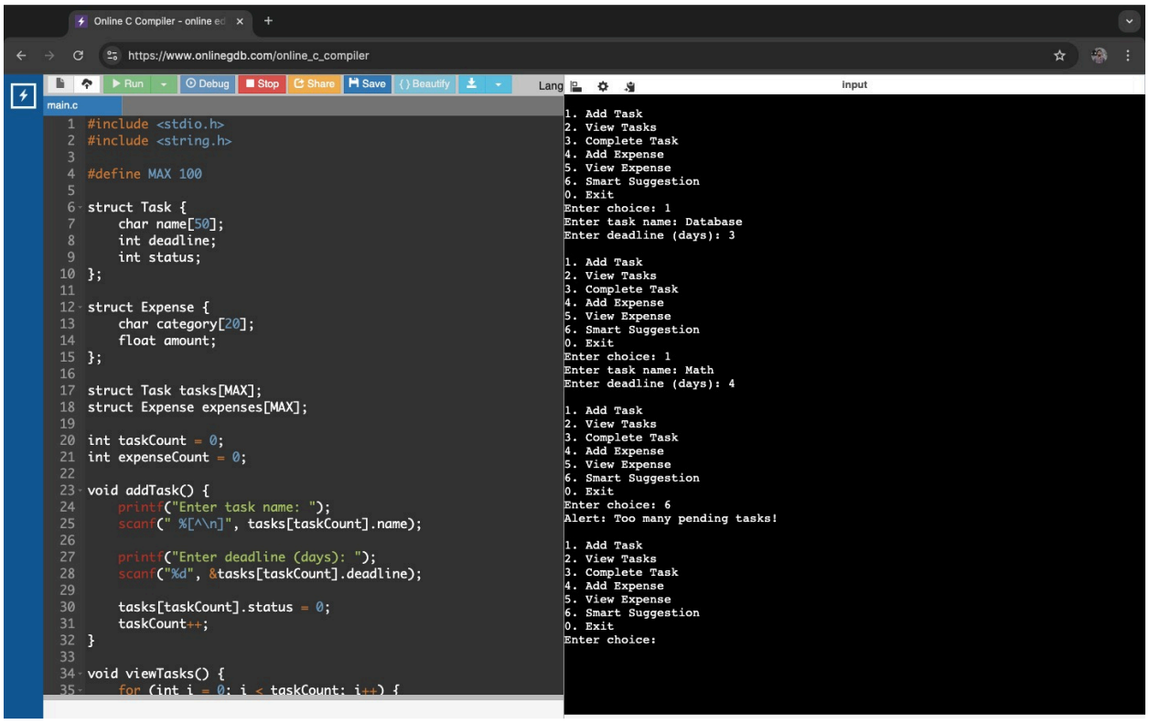
input

```
1. Add Task
2. View Tasks
3. Complete Task
4. Add Expense
5. View Expense
6. Smart Suggestion
0. Exit
Enter choice: 4
Enter category: Food
Enter amount: 6000

1. Add Task
2. View Tasks
3. Complete Task
4. Add Expense
5. View Expense
6. Smart Suggestion
0. Exit
Enter choice: 5
Food: 6000.00
Total Expense: 6000.00

1. Add Task
2. View Tasks
3. Complete Task
4. Add Expense
5. View Expense
6. Smart Suggestion
0. Exit
Enter choice: 6
Warning: You are spending too much!

1. Add Task
2. View Tasks
3. Complete Task
4. Add Expense
5. View Expense
6. Smart Suggestion
0. Exit
Enter choice:
```



8.Appplication

- Useful for students to manage daily activities
- Helps improve productivity
- Assists in basic expense tracking and budgeting

9. Conclusion

This project demonstrates the practical use of basic programming concepts such as structures, arrays, loops, and functions. It shows how a simple software system can solve real-life problems like task management and expense tracking.

Overall, this project is very helpful for beginners to understand how programming can be applied in real-world scenarios.

10. Future Improvements

- Add file handling to store data permanently
- Improve user interface
- Add graphical representation (charts)
- Convert into a web or mobile application

11. Author

Name: Md Tasnimul Islam Shad
Project Title: Student Productivity & Expense Manager
Programming Language: C

12. Learning Outcomes / Course Integration

This project is developed by applying knowledge gained from different professional courses completed on Coursera.

- **Foundations of Project Management by Google**

  → Used in designing the task management system, including task planning, tracking, and deadline handling.
- **Foundations: Data, Data, Everywhere by Google**

  → Applied in collecting, organizing, and analyzing expense data.
- **Introduction to Software Engineering by IBM**

  → Applied in system design, modular coding, and structured development approach.
- **AI Awakening: Implications for the Economy and Society by Stanford University**

  → Used to implement basic rule-based smart suggestions representing simple AI concepts.

This project shows how knowledge from multiple fields can be combined to build a practical and meaningful application.